Guía de Actividades Práctico-Experimentales Nro. 005

1. Datos Generales

Asignatura	Teoría de la programación
Estudiante	Daniela Loreley Alejandro Lozano
Ciclo	1 A
Unidad	**1**
Resultado de aprendizaje de la unidad	Identifica los conceptos fundamentales de la teoría de la programación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	**005**
Tipo	Individual
Título de la Práctica	Del diseño del algoritmo con estructuras secuenciales a la construcción del programa.
Nombre del Docente	José Oswaldo Guamán Quinche
Fecha	Jueves 29 de octubre del 2026 al Viernes 20 de octubre del 2026
Horario	Jueves, 12h30 – 13h30 Viernes, 09h30 - 11h30
Lugar	Aula física asignada al paralelo.
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- Desarrollar la capacidad de transformar un problema en una solución computacional.
- Aplicar estructuras secuenciales en el diseño del algoritmo en un lenguaje de programación.
- Validar la lógica del algoritmo mediante pruebas de escritorio.

3. Materiales y reactivos, Equipos y herramientas

- IDE de programación: Visual Studio Code u otro entorno compatible.
- Lenguaje de programación: Según los contenidos de la unidad.

- Computador personal con sistema operativo Windows, Linux o macOS.
- Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).
- Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF.

- Conexión a internet estable para acceder a recursos digitales y software en línea.

4. Procedimiento / Metodología

Problema:

Inicio

- Presentación del objetivo de la práctica.
- Contextualización del problema: Un estudiante necesita saber qué calificación debe obtener en el tercer certamen (C3) para aprobar la asignatura con una nota final de 60/100 puntos.

  Para calcular la nota final se utilizan las siguientes

  fórmulas: Promedio de certámenes (NC):

$$NC = \frac{C1 + C2 + C3}{3}$$

  Nota final del ciclo (NF):

$$NF = (NC \cdot 0.7) + (NL \cdot 0.3)$$

  Donde:

  - C1 y C2 son las notas de los dos primeros certámenes.
  - C3 es la nota del tercer certamen (la que se debe calcular).
  - NL es la nota de laboratorio.
  - NF es la nota final de la asignatura.

  El programa debe permitir ingresar las notas de C1, C2 y NL; calcular automáticamente la nota mínima necesaria en C3 para que el estudiante apruebe la asignatura.

  Nota: Si el resultado es negativo, significa que ya aprueba con las notas actuales.

Análisis del problema

Datos de entrada: c1, c2, nl, despeje, c3

Proceso:

- **nc** = (c1 + c2 +c3)/3
- **nf** = (nc * 0.7) + (nl * 0.3)
- despeje = 3 * c3 - c1- c2

Resultados esperados: nc_necesario, , c3_minima

Pseudocódigo:

```
Inicio
//Datos de entrada
Definir c1, c2, nl, nc_necesario, c3_minima Como Real
// Entrada de datos
Escribir ("Ingrese la nota de C1:")
Leer c1
Escribir ("Ingrese la nota de C2:")
Leer c2
Escribir ("Ingrese la nota de laboratorio: ")
Leer nl
// Proceso
nc_necesario <- (60 - nl * 0.3) / 0.7
c3_minima <- 3 * nc_necesario - c1 - c2
//Datos de Salida
Escribir ("Nota minima necesaria en C3: " + c3_minima)
Si  (c3_necesario <= 0) ENtonces{
        Imprimir ("El valor es <= 0, por lo que ya apruebas con tus notas
      actuales)
}
Fin
```

5. Resultados

Prueba de escritorio:

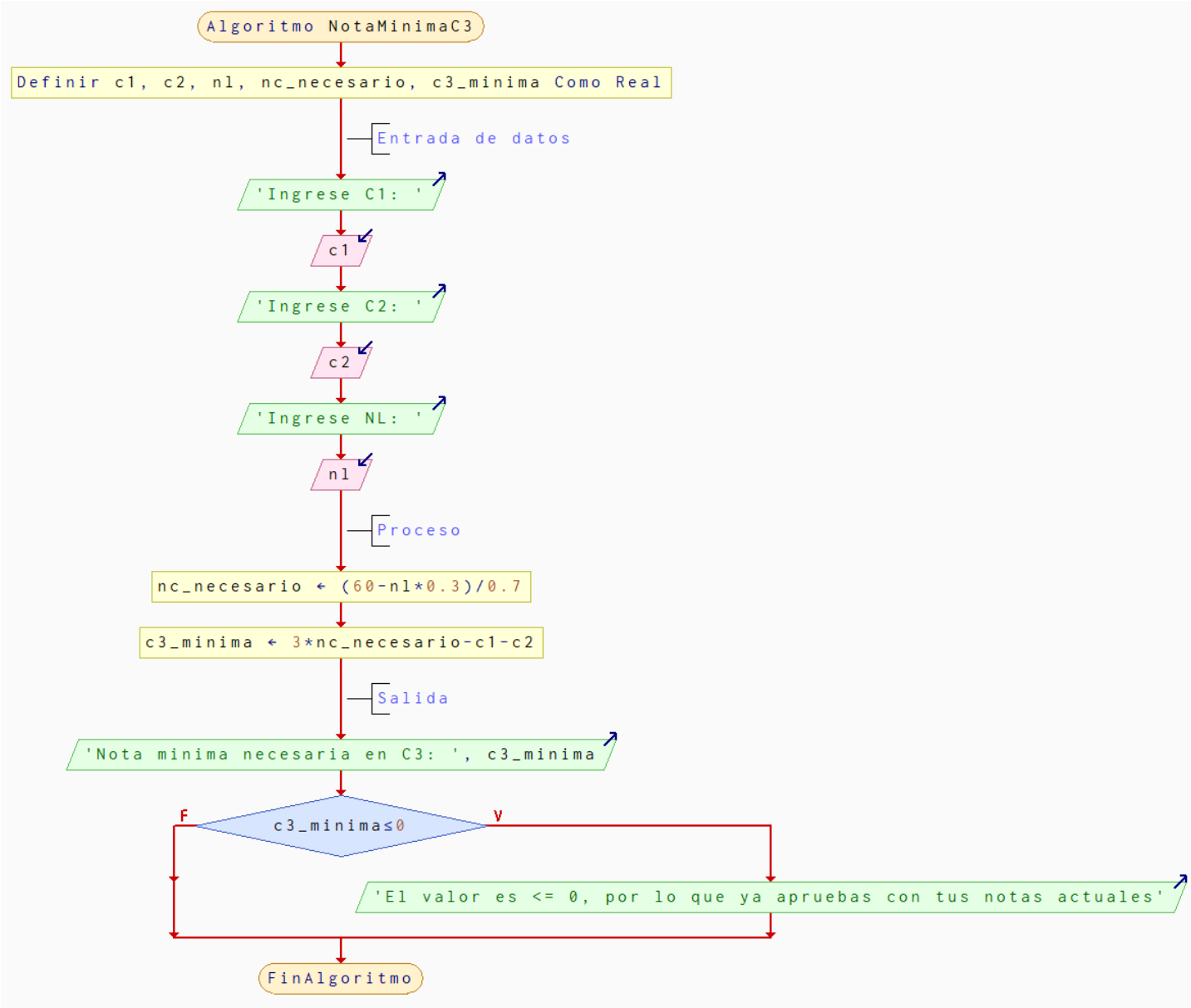
Prueba 1

Paso	c1	c2	nl	nc_necesario	c3_minima	Salida
1	---	---	----	---	---	
2	100	----	---	---	---	Ingrese C1:
3	100	100	---	---	---	Ingrese C1:
4	100	100	100	---	---	Ingrese NL:
5	100	100	100	42.86	---	
6	100	100	100	42.86	-71.43	
7	100	100	100	42.86	-71.43	Nota minima necesaria en C3: -71.43
8	100	100	100	42.86	-71.43	El valor es <= 0, por lo que ya apruebas con tus notas actuales

Prueba 2:

Paso	c1	c2	nl	nc_necesario	c3_minima	Salida
1	---	---	----	---	---	
2	70	----	---	---	---	Ingrese C1:
3	70	20	---	---	---	Ingrese C1:
4	70	20	90	---	---	Ingrese NL:
5	70	20	90	47.14	---	
6	70	20	90	47.14	51.43	
7	70	20	90	47.14	51.43	Nota minima necesaria en C3: 51.43

Diagrama de flujo:



Codificación en C:

```c
#include <stdio.h>
void main (){
    float c1, c2, nl, nc_necesario, c3_minima;
    //entrada de datos
    printf("Ingrese C1: ");
    scanf("%f", &c1);
    printf("Ingrese C2: ");
    scanf("%f", &c2);
    printf("Ingrese NL: ");
    scanf("%f", &nl);
    //proceso
    nc_necesario = (60 - nl * 0.3) / 0.7;
    c3_minima = 3 * nc_necesario - c1 - c2;
    //salida
    printf("Nota minima necesaria en C3: %.2f\n", c3_minima);
    if (c3_minima <= 0) {
        printf("El valor es <= 0, por lo que ya apruebas con tus notas actuales)\n");
    }
    return 0;
}
```

Salida del programa:

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ape5.exe
Ingrese C1: 100
Ingrese C2: 100
Ingrese NL: 100
Nota minima necesaria en C3: -71.43
(Nota: El valor es <= 0, por lo que ya apruebas con tus notas actuales)
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ape5.exe
Ingrese C1: 70
Ingrese C2: 20
Ingrese NL: 90
Nota minima necesaria en C3: 51.43
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

6. Preguntas de Control:

- **¿Qué es un programa con estructuras secuenciales?**

  Un programa con estructuras secuenciales es aquel donde las instrucciones se ejecutan de forma lineal y consecutiva. Las instrucciones se ejecutan una tras otra. Se sigue el orden exacto del código de inicio a fin, el programa es continuo. No presenta saltos, bifurcaciones y repeticiones de bucles.

- **¿Cómo se traslada un algoritmo en pseudocódigo o diagrama de flujo a un lenguaje de programación?**

Después de realizar el análisis del problema, se crea el algoritmo con pseudocódigo y diagrama de flujo y se comprueba la lógica mediante una prueba de escritorio. Luego se procede a la fase de codificación. En la codificación se traduce de forma sistemática el algoritmo a la sintaxis y semántica de un lenguaje de programación, como en este caso C.

- **¿Por qué es importante validar el programa mediante casos de prueba?**

  Validar un programa mediante casos de prueba es fundamental en la resolución algorítmica porque permite verificar de manera sistemática si las instrucciones, fórmulas y procesos generan exactamente los datos de salida esperados y responden a la lógica del problema planteado, además ayuda a identificar fallos puntuales en el orden de las instrucciones, en las asignaciones de variables o en las condiciones de ejecución, facilitando la corrección del código antes de darlo por finalizado

7. Conclusiones

- Se logró desglosar y solucionar el problema planteado mediante el despeje algebraico de las fórmulas de evaluación académica, permitiendo determinar con precisión el puntaje mínimo requerido en el tercer certamen para aprobar la asignatura con una calificación final de 60/100.
- Se evidenció la utilidad de las estructuras de control secuenciales para procesar datos de manera lineal y continua (lectura de $C1$, $C2$ y NL, cálculo de $nc_necesario$ y $c3_minima$, y visualización del resultado).
- La realización de la prueba de escritorio y la posterior ejecución del código en el lenguaje C en el entorno Visual Studio Code permitieron verificar la coherencia lógica de las fórmulas empleadas y la precisión de los resultados esperados ante distintos escenarios de entrada.

8. Recomendaciones

- Incorporar estructuras condicionales que restrinjan el ingreso de calificaciones únicamente en el rango permitido (por ejemplo, de 0 a 100), evitando así errores de cálculo derivados de valores fuera de escala o negativos introducidos por error.
- Limitar el rango de salida de la nota calculada: Es conveniente formatear la salida algorítmica de modo que, si el cálculo de $c3_minima$ supera la nota máxima posible (100 puntos), el programa alerte al usuario de que la aprobación ya no es alcanzable con el tercer certamen bajo las notas actuales.

9. Bibliografía / Referencias

[1] Remington, C. U., & Martínez, M. M. A. (2023). Lógica de programación con Pseint. Enfoque práctico. En *Fondo Editorial Remington eBooks*. https://doi.org/10.22209/9786289585247

[2] Bonilla, M. T. (2023). Algoritmos y tipos de datos en C. En *Editorial Universidad de Sevilla eBooks*. https://doi.org/10.12795/9788447225088

[3] Surra, B. (2023, 20 julio). *Por qué realizar pruebas de validación de software*. MyTaskPanel Consulting. https://www.mytaskpanel.com/pruebas-de-validacion-de-software/

10. Anexos

Código de git hub:
https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/APE5/ape5.c

Criterio	Puntos máximos
Desarrollo del contenido temático 1. Describe el tema solicitado con claridad, pertinencia, coherencia y suficiencia, evidenciando comprensión de los conceptos abordados.	2 ptos
Evidencia práctica o aplicación 6. Presenta productos, procedimientos o resultados acordes con la actividad planteada, demostrando cumplimiento y relación con el contenido desarrollado.	6 ptos
Reflexión crítica ● Formula reflexiones pertinentes sobre el aprendizaje alcanzado, destacando la importancia del tema y su aporte al proceso formativo.	1 pto
Presentación y formato ● Estructura del informe, redacción clara y ortografía. ● Bibliografía en IEEE >= 2020.	1 pto
Total	10 pto

11. Elaboración y Aprobación

Elaborado por	José Oswaldo Guamán Quinche **Docente**	

Aprobado por	Edison Leonardo Coronel Romero. **Director de Carrera**	